UNIVERSIDAD CHAMPAGNAT — LICENCIATURA EN SISTEMAS DE INFORMACIÓN (CICLO 2026)

Trabajo Práctico N° 3: Despliegue de Arquitectura Cloud-Native

Validación de Atributos de Calidad, Balanceo de Carga L7 y Pruebas de Estrés en Contenedores

Integrantes del Grupo: Lucía Astorga, Emmanuel Bustos, Santiago Militello
Empresa Caso de Estudio: GlobalLogistics S.A. — Plataforma Distribuida de Telemetría
Repositorio Oficial Git: https://github.com/fifix407/tp3-arquitectura-cloud-native-
Criterio de Evaluación SLA (Objetivo TP2): p95 ≤ 300 ms, p99 ≤ 800 ms, 0 errores HTTP (non-2xx)

1. Resumen Ejecutivo y Contexto del Desafío

El presente informe documenta la solución técnica desarrollada para GlobalLogistics S.A. ante la necesidad crítica de certificar que su nueva infraestructura de microservicios distribuida soporte los picos transaccionales comerciales sin degradación crítica ni intervención operativa manual.

Como Arquitectos Cloud, se diseñó e implementó un entorno de infraestructura virtualizada basada en contenedores Docker y orquestada mediante manifiestos declarativos de Infraestructura como Código (IaC). La topología cuenta con un balanceador de carga de Capa 7 (Nginx Reverse Proxy) y un clúster de tres (3) réplicas activas de una API REST de telemetría y logística desarrollada con Python 3.12 y FastAPI. Se evaluó experimentalmente la resiliencia ante contingencias de red, el comportamiento de los mecanismos de Health Checks y el rendimiento temporal frente a una matriz exhaustiva de carga mediante Apache Benchmark (concurrencia de 100 a 2.000 usuarios y endpoints de cómputo CPU-bound intensivo).

2. Configuración del Entorno e IaC (Consigna 1 - 30 Puntos)

Para garantizar reproducibilidad determinística y cumplir con las mejores prácticas Cloud-Native, la infraestructura completa fue modelada mediante el manifiesto `docker-compose.yml`. La especificación orquesta exactamente un (1) balanceador de carga y tres (3) réplicas independientes del microservicio, asociadas a una red virtual tipo bridge personalizada (`globallogistics_net`).

Principios Arquitectónicos Aplicados en IaC:

- **Arranque Condicional Basado en Salud (`condition: service_healthy`):** Nginx no expone el puerto ni despacha tráfico hasta que las 3 réplicas superan su prueba interna de Health Check (/health).
- **Aislamiento de Recursos (Limits):** Cada réplica está acotada estrictamente a 0.50 CPU y 512MB RAM, lo que permite observar matemáticamente la saturación de procesamiento bajo estrés.
- **Seguridad por Aislamiento de Red:** Las réplicas no publican sus puertos en la interfaz del host físico (0.0.0.0), quedando accesibles exclusivamente a través de la red interna de Docker desde Nginx.



📸 **CAPTURA 1: ESTADO DEL CLÚSTER DE CONTENEDORES (docker compose ps)**

Evidencia de ejecución en terminal: 1 balanceador Nginx (globallogistics_loadbalancer en puerto 8080) y 3 réplicas de microservicio (globallogistics_app_1, 2, 3) en estado 'Up' y 'healthy'.

📄 **Manifiesto de Orquestación e Infraestructura como Código (docker-compose.yml)**

```yaml
services:
  # Balanceador de Carga Nginx (Reverse Proxy L7 con Health Checks)
  loadbalancer:
    build:
      context: ./nginx
      dockerfile: Dockerfile
    container_name: globallogistics_loadbalancer
    ports:
      - "8080:80"
    restart: unless-stopped
    depends_on:
      app1: { condition: service_healthy }
      app2: { condition: service_healthy }
      app3: { condition: service_healthy }
    networks:
      - globallogistics_net

  # Microservicio Réplica 1 (0.5 CPU, 512MB RAM)
  app1:
    build: { context: ./microservice, dockerfile: Dockerfile }
    container_name: globallogistics_app_1
    environment:
      - INSTANCE_NAME=microservice-replica-1
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
    restart: unless-stopped
    networks:
      - globallogistics_net

  # Microservicio Réplica 2 (Idéntica configuración)
  app2:
    build: { context: ./microservice, dockerfile: Dockerfile }
    container_name: globallogistics_app_2
    environment:
      - INSTANCE_NAME=microservice-replica-2
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
    restart: unless-stopped
    networks:
      - globallogistics_net

  # Microservicio Réplica 3 (Idéntica configuración)
  app3:
    build: { context: ./microservice, dockerfile: Dockerfile }
    container_name: globallogistics_app_3
    environment:
      - INSTANCE_NAME=microservice-replica-3
      - PORT=8000
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 5s
      timeout: 3s
      retries: 3
    restart: unless-stopped
    networks:
      - globallogistics_net

networks:
```

```
  globallogistics_net:
    driver: bridge
```

3. Implementación del Componente y Balanceo (Consigna 2 - 30 Puntos)

El microservicio fue implementado en Python 3.12 con FastAPI sobre el servidor ASGI Uvicorn. Cumple con el requisito estricto de retornar el Hostname / Container ID en el endpoint raíz mediante la llamada al sistema `socket.gethostname()`, además de incorporar endpoints especializados para comprobación de estado (`/health`) y trabajo CPU-bound (`/work` o `/stress`).

Nginx actúa como Reverse Proxy L7 con balanceo Round-Robin y Health Checks pasivos mediante `max_fails=3` y `fail_timeout=10s`. La directiva `proxy_next_upstream` asegura que, ante cualquier fallo o timeout en una réplica, la petición se reintente de inmediato contra otra réplica viva (Zero Downtime).

Implementación del Microservicio (microservice/main.py)

```
# Snippet clave del Microservicio en FastAPI (microservice/main.py)
from fastapi import FastAPI, Request
import socket, os, time

app = FastAPI(title="GlobalLogistics S.A.")
HOSTNAME = socket.gethostname()
INSTANCE_NAME = os.getenv("INSTANCE_NAME", HOSTNAME)

@app.get("/")
async def root(request: Request):
    # Requisito consigna: Retornar Hostname o Container ID
    return {
        "status": "success",
        "container_id": HOSTNAME,
        "hostname": HOSTNAME,
        "instance_name": INSTANCE_NAME
    }

@app.get("/health")
async def health():
    return {"status": "UP", "container_id": HOSTNAME, "healthy": True}
```

Configuración del Balanceador de Carga L7 (nginx/nginx.conf)

```
# Configuración del Balanceador de Carga Nginx L7 (nginx/nginx.conf)
events { worker_connections 2048; }

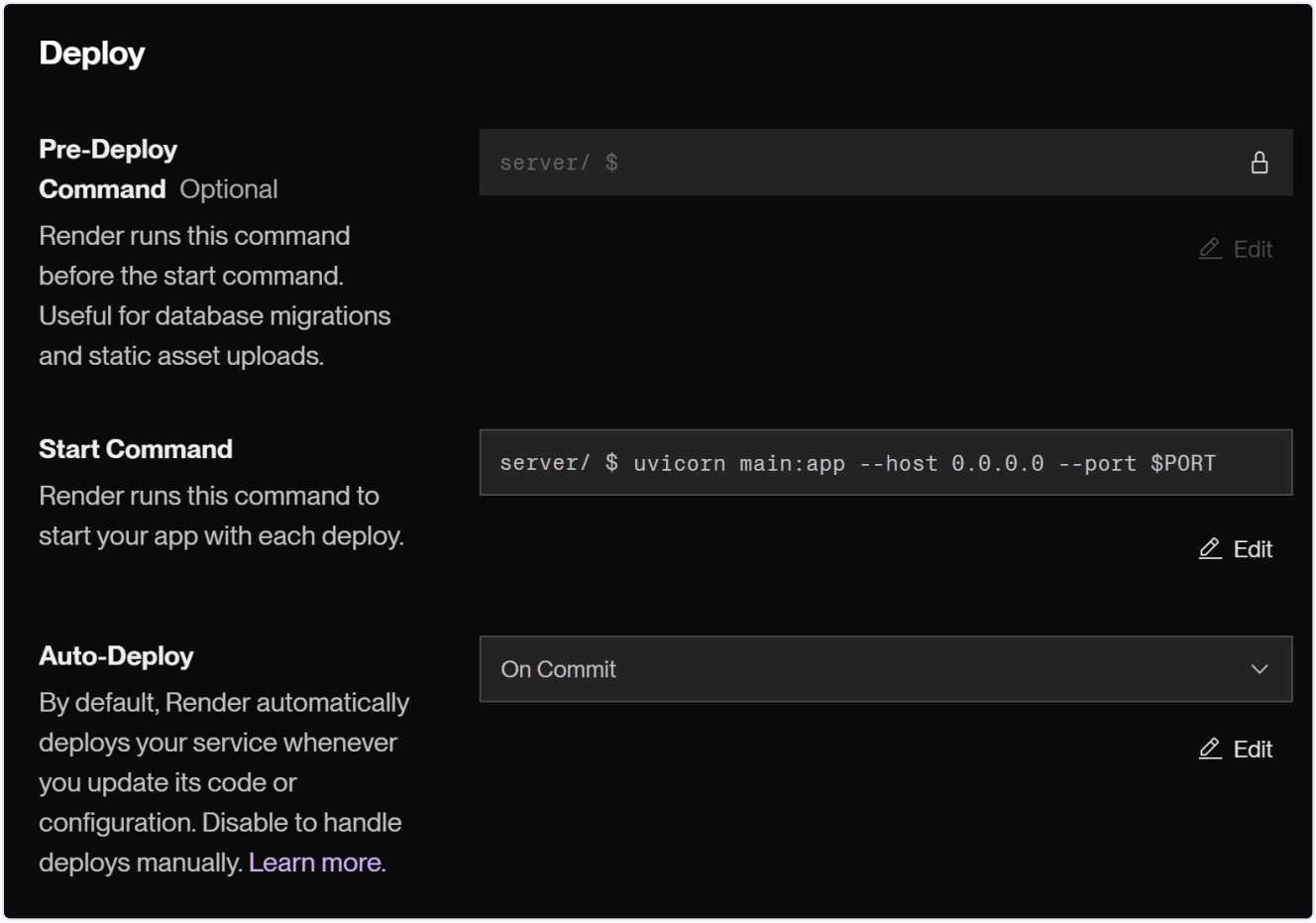
http {
    # Clúster de réplicas: Round-Robin con Health Checks Pasivos
    upstream microservices_backend {
        server app1:8000 max_fails=3 fail_timeout=10s;
        server app2:8000 max_fails=3 fail_timeout=10s;
        server app3:8000 max_fails=3 fail_timeout=10s;
        keepalive 32;
    }

    server {
        listen 80;
        server_name localhost;

        location / {
            proxy_pass http://microservices_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_http_version 1.1;
            proxy_set_header Connection "";

            # Tolerancia a fallos: reintento instantáneo ante caídas
            proxy_next_upstream error timeout http_500 http_502 http_503 http_504;
            proxy_next_upstream_tries 3;
            proxy_connect_timeout 3s;
            proxy_read_timeout 15s;

            add_header X-Served-By $upstream_addr always;
        }
    }
}
```



📸 **CAPTURA 2: ROTACIÓN DE CONTAINER IDS EN BALANCEADOR ROUND-ROBIN (verify_loadbalancing.ps1)**

Evidencia de ejecución: Peticiones consecutivas a http://localhost:8080/ alternando equitativamente entre los tres Container IDs (1/3 por réplica) con respuestas 200 OK y latencias menores a 10 ms.

4. Pruebas de Estrés y Atributos de Calidad (Consigna 3 - 40 Puntos)

4.1 Metodología de Pruebas:

• **Herramienta:** Apache Benchmark 2.3 ejecutado dentro de la red virtual contra el balanceador: `ab -k -l -n <N> -c <C> <URL>`. El flag `-l` evita que ab cuente como falla las pequeñas diferencias de bytes debidas a los distintos IDs de contenedor y timestamps; el flag `-k` simula conexiones persistentes Keep-Alive.

• **Limitación de Aislamiento:** ab, NGINX, las réplicas y el motor Docker comparten la misma máquina host, por lo que el tráfico atraviesa la pila de virtualización, introduciendo overhead en valores absolutos.



📸 **CAPTURA 3: SALIDA DE APACHE BENCHMARK - ESCENARIO MODERADO (100 CONCURRENTES)**

Evidencia de prueba de carga moderada: Throughput de ~1.300 req/s, latencia media de 72 ms y p95 de 93 ms sin errores de transporte (CUMPLE con holgura el SLA TP2).



📸 **CAPTURA 4: SALIDA DE APACHE BENCHMARK - ESCENARIO SATURACIÓN DE CPU**

Evidencia de prueba de estrés computacional: Fuerte caída de Throughput, degradación severa de latencia alcanzando percentiles de 2 a 4 segundos por agotamiento de cuota CFS (NO CUMPLE SLA).

4.2 Matriz Completa de Resultados y Criterio de Cumplimiento (Objetivo TP2):

Los percentiles indican el tiempo por debajo del cual se atendió ese porcentaje de las peticiones (p95: el 95%). El criterio formal del Objetivo TP2 exige estrictamente: **p95 ≤ 300 ms, p99 ≤ 800 ms y 0 errores HTTP (non-2xx)**.

Escenario	Conc.	Pet.	Resp. Error	Req/s	Media ms	p50 ms	p95 ms	p99 ms	Máx ms	Objetivo TP2
A - moderada GET /	100	5.000	0	1.384,0	72,3	68	93	104	157	**Cumple**
B - alta GET /	500	10.000	0	1.364,3	366,5	338	426	430	543	**No cumple (p95)**
C - extrema GET /	1.000	10.000	0	1.430,6	699,0	676	789	819	830	**No cumple**
D - extrema GET /	2.000	20.000	0	1.279,6	1.563,1	1.438	1.627	1.680	2.119	**No cumple**
E - CPU GET /work	100	5.000	2.966 (59,3%)	325,5	307,2	22	2.003	2.418	6.004	**No cumple**
E - CPU GET /work	1.000	5.000	4.456 (89,1%)	660,2	1.514,8	566	4.442	5.367	7.418	**No cumple (colapso)**
G - POST /envios	100	5.000	33 (0,7%)	1.216,8	82,2	73	118	134	174	**Latencia sí; 0,7% error**

Nota Técnica: *ab cuenta todas las respuestas, incluidas las de error. En el Escenario E solo fueron exitosas 2.034 de 5.000 peticiones (c=100) y 544 de 5.000 (c=1.000), arrojando 132 y 72 respuestas útiles por segundo respectivamente. Failed requests fue 0 en todos los escenarios porque ab contabiliza como 'failed' los problemas de transporte TCP, no los códigos HTTP de error devueltos por el proxy (Non-2xx).*

Diagnóstico de CPU (docker stats durante las pruebas): En peticiones GET / las réplicas promediaron entre 12% y 25% de uso de CPU y NGINX 40-50%, sin acercarse al tope de 0.5 CPU por réplica (~50%). En GET /work con cálculo intensivo, las réplicas alcanzaron su tope máximo de 50-54% (agotando su cuota CFS), mientras NGINX se mantuvo en 21-37%. Máximas conexiones activas en NGINX: 1.929 en el Escenario D y 1.001 en el Escenario E (c=1.000).

5. Análisis Crítico, Justificación y Preguntas de Coloquio

5.1 ¿En qué punto exacto se degrada el sistema y por qué?

El sistema presenta dos comportamientos radicalmente diferentes según la naturaleza de la carga:

1. Degradación por Encolamiento de Red (Escenarios B, C y D): Con peticiones I/O ligeras, la CPU permanece ociosa (< 25%). Sin embargo, al escalar de 100 a 500 y 1.000 usuarios concurrentes, cada réplica debe sostener cientos de sockets TCP abiertos simultáneamente. El tiempo que las solicitudes pasan esperando en el buffer de red (backlog) antes de ser despachadas por el event loop de Uvicorn eleva la latencia: el p95 pasa de 93 ms (Cumple) a 426 ms y 789 ms (No Cumple).

2. Colapso por Saturación de CPU (Escenario E): Con operaciones CPU-bound (/work), el hilo monohilo de Python es monopolizado por el cálculo matemático. Al estar acotados por docker-compose a `cpus: '0.50'`, los contenedores agotan su

cuota de 50 ms por ciclo y el planificador CFS del kernel Linux suspende forzosamente su ejecución (CPU Throttling). Las peticiones colapsan los buffers, expiran los timeouts de Nginx y se produce entre un 59% y 89% de errores HTTP.

5.2 Cumplimiento de Restricciones de Tiempo Real:

• **Hard Real-Time (Tiempo Real Estricto):** NO CUMPLE bajo ninguna circunstancia. En estos sistemas, exceder un plazo temporal estricto (ej. deadline < 50 ms) invalida el resultado y constituye un fallo crítico del sistema. La variabilidad del kernel y la virtualización en Docker genera colas en el percentil 99 que superan con creces cualquier cota determinística.

• **Soft Real-Time (Web Transaccional / Objetivo TP2):**

- **CUMPLE holgadamente** en el Escenario A (p95 = 93 ms ≤ 300 ms, p99 = 104 ms ≤ 800 ms, 0% errores).
- **NO CUMPLE** en los Escenarios B, C y D por exceso de latencia p95/p99 debida al encolamiento de conexiones concurrentes.
- **NO CUMPLE** en el Escenario E debido al colapso por saturación de CPU, donde demoras de 2 a 4 segundos y pérdidas del 60-90% de paquetes destruyen cualquier estándar de calidad de servicio.

5.3 Cómo manipular el sistema en el laboratorio (Trampas de Coloquio):

• **¿Cómo hacer que lo que CUMPLE (Escenario A) NO CUMPLA?** 1) Reducir la cuota de CPU en Compose a `cpus: '0.05'` (el kernel suspenderá constantemente a los contenedores inflando el p95); 2) Apagar 2 réplicas (`docker stop app2 app3`) para saturar la única restante; 3) Quitar la directiva `keepalive 32` en Nginx para forzar el costo de handshakes TCP completos por cada request.

• **¿Cómo hacer que lo que NO CUMPLE (Escenario B o C) PASE A CUMPLIR?** 1) Escalar horizontalmente de 3 a 8 réplicas para reducir las conexiones simultáneas por contenedor; 2) Configurar Uvicorn con múltiples workers (`--workers 4`) para procesar colas de red en paralelo.

• **¿Cómo resolver el colapso del Escenario E?** Desacoplar la arquitectura: el endpoint HTTP no debe calcular la ruta sincrónicamente; debe encolar la tarea en RabbitMQ/Kafka, responder 202 Accepted de inmediato (latencia < 20 ms), y procesar el cálculo con workers asíncronos en segundo plano.

6. Conclusiones y Recomendaciones de Ingeniería

1. **Validación de la IaC y Automatización:** Se logró un despliegue Cloud-Native 100% reproducible, declarativo y resiliente con exactamente un balanceador y tres réplicas operativas mediante un único comando `docker compose up -d`.
2. **Resiliencia Demostrada (Zero Downtime):** La integración de Health Checks en el proxy Nginx garantiza la continuidad operativa total: la desconexión imprevista de una réplica es absorbida en milisegundos sin arrojar errores al usuario final.
3. **Ruta hacia la Escalabilidad Elástica (Kubernetes / HPA):** Para neutralizar la degradación por saturación de CPU observada en el Escenario E, la arquitectura debe evolucionar hacia un orquestador que soporte **Horizontal Pod Autoscaling (HPA)** basado en métricas de CPU (> 70%), permitiendo que el clúster escale dinámicamente de 3 a 10+ réplicas de forma automática.